

Day 12 lab: local decisions and MQTT

Naeemullah Khan

naeemullah.khan@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

October 7, 2026

- ▶ **Goal:** a system of two boards that decides locally and sends its results to the cloud, also when the link to the cloud fails.
- ▶ **Boards:** the XIAOML Kit and the Raspberry Pi 5. The instructor laptop is the “cloud”.
- ▶ **Deliverable:** a live demonstration with the link to the cloud connected and cut, and the file `report.md` with the topic tree, the measured values, and the Decision Log.

From the lecture to the lab

The lecture gave the decision layer with a state machine, the MQTT topics, the QoS levels, the last will, and store and forward. Its values came from one computer with two brokers. Today the messages cross the Wi-Fi, the keyword model of Day 5 makes the events, and you cut the link yourself.

Key point: The local action must work with no cloud, and no message may be lost when the cloud comes back.

Plan of the three hours



Part	Work	Time
A	Broker: Mosquitto on the Raspberry Pi, publish and subscribe from the command line	30 min
B	Telemetry: IMU readings with MicroPython, keyword events with the Arduino sketch	45 min
C	Local decision: a state machine on the Raspberry Pi, a command back to the XIAO	40 min
D	Offline operation: store and forward to the cloud, a cut of the link	35 min
Check by the instructor		30 min

Write each result in `report.md` when you get it.

Hardware for each group

- ▶ XIAOML Kit and a USB-C cable.
- ▶ Raspberry Pi 5 with the active cooler, the card of your group, and the 27 W power supply.
- ▶ Laptop in the lab network `edgeai-lab`.
- ▶ For the class: the instructor laptop with a Mosquitto broker at `X.10`.

Software

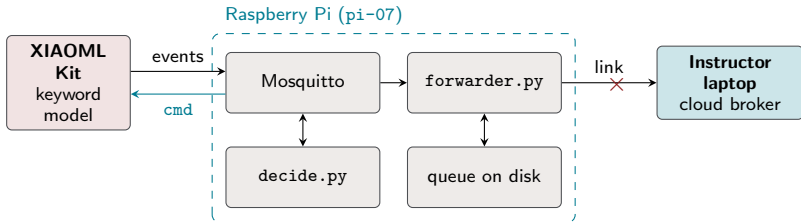
- ▶ Card: Mosquitto and its clients, the environment `~/yolo` with `paho-mqtt`.
- ▶ Laptop: Python 3 with `paho-mqtt 2.x`, the Arduino IDE with the `esp32 core 3.3.12`, `PubSubClient 2.8`, `U8g2`.
- ▶ The ZIP library of your Day 5 keyword project.
- ▶ MicroPython `v1.29.0`, `esptool`, `mpremote (HW-01)`.

Lab files

`Labs/day12/` on the laptop, and a copy in `~/edgeai/` on the Raspberry Pi.

Names

`gNN` is your group: `pi-NN`, `X.(100+NN)`. The examples use `g07` and `X = 192.168.8`.



- ▶ The XIAO, the broker, and `decide.py` form the local system. The LED command needs no link to the laptop.
- ▶ `forwarder.py` writes each decision and each summary to a queue on the disk, and sends it to the cloud broker with QoS 1.
- ▶ On the group laptop, `cloud_check.py` counts the messages of the group on the cloud broker.

Topic	From	Payload	QoS, retained
edgeai/g07/xiao/imu	XIAO (B1)	one IMU reading each 200 ms	0, no
edgeai/g07/xiao/result	XIAO	one keyword event	0, no
edgeai/g07/xiao/stats	XIAO	each 10 s: windows, events, memory	0, no
edgeai/g07/xiao/status	XIAO	online, will offline	1, yes
edgeai/g07/xiao/cmd	Raspberry Pi	led=1, led=0	1, no
edgeai/g07/pi/decision	Raspberry Pi	one change of state	1, no
edgeai/g07/pi/summary	Raspberry Pi	each 10 s: the state and the counts	1, no
edgeai/g07/pi/status	Raspberry Pi	online, will offline	1, yes

The forwarder sends pi/decision, pi/summary, and xiao/stats to the cloud.
Each message with JSON has a sequence number seq.

Step	Work	Time
1	Copy the files, start the broker for the lab network	10 min
2	Publish and subscribe, wildcards	10 min
3	QoS, retained message, last will	10 min

```
scp -r ../day12 ../hardware/HW-07 edge@pi-07.local:edgeai/ # laptop
sudo cp ~/edgeai/HW-07/mosquitto_lab.conf /etc/mosquitto/conf.d/lab.conf
sudo systemctl restart mosquitto && ss -tln | grep 1883 # Pi
mosquitto_sub -h localhost -t 'edgeai/g07/#' -v # terminal 1
mosquitto_pub -h pi-07.local -t edgeai/g07/test/a -m hello # terminal 2
```

- ▶ Copy lab.conf only if it is not in the folder. With no listener line, Mosquitto 2 accepts only local clients.
- ▶ **You see:** ss shows 0.0.0.0:1883. Terminal 1 shows edgeai/g07/test/a hello.
- ▶ **You record:** which of the three test messages each filter gets: edgeai/g07/# and edgeai/+/test/a.

```
mosquitto_pub -h pi-07.local -t edgeai/g07/test/q -m x -q 2 -d # -q 1, -q 0
mosquitto_pub -h pi-07.local -t edgeai/g07/test/state -m on -r
mosquitto_sub -h pi-07.local -t edgeai/g07/test/state -v -C 1
mosquitto_pub -h pi-07.local -t edgeai/g07/test/state -n -r # delete it
mosquitto_sub -h pi-07.local -t edgeai/g07/none -k 5 \
    --will-topic edgeai/g07/laptop/status --will-payload offline &
kill -9 %1
```

- ▶ -d prints each packet of the client:

```
Client (null) sending PUBLISH (d0, q2, r0, m1, 'edgeai/g07/test/q', ...
Client (null) received ...
```

- ▶ **You record:** the packets of each QoS level, whether the new subscriber gets the retained message, and the time from kill -9 to offline in terminal 1.
- ▶ Compare with the lecture: which of the three ends of a connection is this?

1. Write the MicroPython firmware: steps 1 to 3 of `../hardware/HW-01/README.md`, as on Day 2.
2. Write the Wi-Fi password, the address of your Raspberry Pi, and your group in `../hardware/HW-07/micropython/config.py`.

```
mpremote connect PORT fs cp ../hardware/HW-01/board/lsm6ds3.py :lsm6ds3.py
mpremote connect PORT fs cp ../hardware/HW-07/micropython/config.py :config.py
mpremote connect PORT run ../hardware/HW-07/micropython/mqtt_imu.py
python3 ../hardware/HW-07/host/mqtt_check.py --broker pi-07.local \
--group g07 --seconds 30
```

- ▶ **You see:** a table with one line for `edgeai/g07/xiao/imu` and a `RESULT` line. The nominal rate is 5 messages each second.
- ▶ **You record:** the messages, the lost messages, the rate. Then remove the USB cable for 10 s, connect it again, and record the gap in seq.

Part B: keyword events with the sketch (25 min)

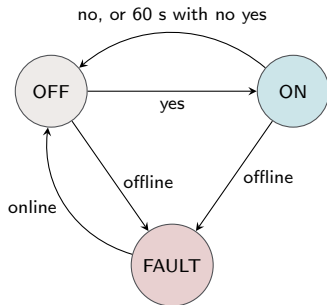
1. Open `sketches/kws_mqtt/kws_mqtt.ino`. Add the ZIP library of your Day 5 project, and change the `#include` line to its header.
2. In `arduino_secrets.h`: the Wi-Fi password, the address of your Raspberry Pi, your group.
3. Board XIAO_ESP32S3, Tools > PSRAM > OPI PSRAM, Erase All Flash Before Sketch Upload > Enabled. Upload, then set the erase option back to Disabled.

```
MQTT: connecting to 192.168.8.107 ... connected
MQTT sent: {"seq":1,"ms":...,"label":"yes","score":...}
```

- ▶ Say “yes” and “no”. Terminal 1 of Part A shows the events on `edgeai/g07/xiao/result`, and one stats message each 10 s.
- ▶ Send `led=1` by hand to `edgeai/g07/xiao/cmd` with `-q 1`: the LED goes on.
- ▶ **You record:** the events for 10 spoken words of each keyword.

Source: The values of the lines come from your board.

Part C: Task C1, the state machine (15 min)



- ▶ Complete `next_state` in `pi/decide.py`. The docstring gives the six rules, in their order.
- ▶ A keyword below the threshold (0.8) changes nothing. In **FAULT**, only “online” changes the state.
- ▶ The program checks your function with a table of 12 transitions.

```
scp pi/decide.py edge@pi-07.local:edgeai/day12/pi/ # laptop
cd ~/edgeai/day12/pi && ~/yolo/bin/python decide.py --group g07 # Pi
```

You see: Task C1: complete as the first line.

Part C: run the decision (25 min)

Task C1: complete

Settings: threshold 0.80, light on for 60 s after a yes

event yes score ... seq ...

state OFF -> ON yes ... command led=1 (... ms)

- ▶ The time in brackets is the local time on the Raspberry Pi, from the event to the command.
- ▶ Say “yes”: the LED goes on. Say “no”: it goes off. Say “yes” and wait 60 s: it goes off.
- ▶ **You record:** some local times, and the delay from the word to the LED as you see it.
- ▶ **A failed part (10 min):** remove the USB cable of the XIAO. Record the seconds until -> FAULT status offline. Connect the cable again: the state goes to OFF.

Source: The values “...” come from your boards.

1. **Task D1 (5 min):** complete `count_sequence` in `cloud_check.py` on the laptop.
2. **Start the two programs (10 min).** Keep `decide.py` running.

```
cd ~/edgeai/day12/pi                                     # second SSH terminal
~/yolo/bin/python forwarder.py --group g07 --cloud 192.168.8.10 --reset
python3 cloud_check.py --broker 192.168.8.10 --group g07 --seconds 600
```

- ▶ **You see** on the Raspberry Pi Cloud broker: connected, then one line each 5 s:

queue	0	stored	12	forwarded	12	link up
-------	---	--------	----	-----------	----	---------

- ▶ On the laptop: Task D1: complete, then one line for each message of the group on the cloud broker.

Part D: cut and restore the link (20 min)

```
bash ~/edgeai/day12/pi/link.sh down 192.168.8.10      # third SSH terminal
bash ~/edgeai/day12/pi/link.sh up 192.168.8.10        # after 3 minutes
```

- ▶ down adds a blackhole route: the Raspberry Pi drops each packet to the cloud broker, with no answer. All other traffic continues.
- ▶ During the cut, say “yes” and “no”: the LED must follow each word. The forwarder prints `link lost` after the keep-alive finds the cut, and the queue grows.
- ▶ After up: Cloud broker: connected, the queue goes to 0, and `cloud_check.py` marks the old messages with `late ... s`.
- ▶ Stop `cloud_check.py` with `Ctrl-C`. It prints a table for each topic and the result line.

```
topic                received  unique  lost double
RESULT: ...
```

You record: the seconds until `link lost`, the queue each minute, the seconds until the queue is 0, the largest `late`, the table.

The instructor checks this at the end of the lab:

- ▶ ✓ `decide.py` prints Task C1: complete, and `cloud_check.py` prints Task D1: complete.
- ▶ ✓ Link connected: a spoken “yes” turns the LED on, “no” turns it off, and the decision appears on the cloud broker.
- ▶ ✓ Link cut: the LED follows the keywords, and the queue of the forwarder grows.
- ▶ ✓ After the link returns, `cloud_check.py` prints RESULT: PASS – no lost message.
- ▶ ✓ The Decision Log gives numbers and names one trade-off.

Hand in: the file `report.md` with the topic tree and the table of `cloud_check.py`.

A cold store has five temperature sensors, a door contact, and a fan. The cloud is behind a mobile modem, which loses the link for up to 2 hours each day.

- ▶ Which decisions stay on the device?
- ▶ What does each device send, how often, with which QoS, and which messages are retained?
- ▶ How large must the queue of the forwarder be, and what happens when it is full?

A good Decision Log (about 100 words):

- ▶ gives your measured message rate, the queue length during your cut, and the time until the queue was empty,
- ▶ calculates the queue for 2 hours from these numbers,
- ▶ names what you lose: for example detail of the data against the size of the queue.

If a part does not work, and common problems

Problem	Fix or fallback
Connection refused from the laptop	Copy <code>lab.conf</code> (step A1) and restart Mosquitto
The XIAO prints failed, <code>rc=-2</code>	Use the IP address of the Raspberry Pi in <code>arduino_secrets.h</code>
The Day 5 library is not on the laptop	Upload the HW-07 sketch <code>mqtt_imu</code> , or send events by hand with <code>mosquitto_pub</code>
<code>decide.py</code> changes nothing	Task C1, or a score below 0.8. Check the first line.
Lost messages after a restart of <code>decide.py</code>	The sequence starts again at 1: start <code>cloud_check.py</code> again
<code>link.sh</code> reports an error	The instructor stops the cloud broker for 3 minutes
The cloud broker does not answer	Use the Raspberry Pi of a second group: <code>--cloud pi-MM.local</code>
A task is not complete in time	Use the file of <code>solutions/</code> , and write this in the report

The README of the lab has the complete list.

- ▶ *TinyML Made Easy: XIAO ESP32S3* by Marcelo Rovai.
github.com/Mjrovai/XIAO-ESP32S3-Sense, Apache-2.0.
- ▶ *XIAO: Big Power, Small Board* by Lei Feng and Marcelo Rovai.
github.com/Mjrovai/XIAO_Big_Power_Small_Board-ebook, GPL-3.0.